

راهنمای کاربردی SQLite3

زبان: فارسی

سبک: آموزشی و قابل فهم برای مبتدی تا متوسط

1. معرفی SQLite

SQLite چیست؟

SQLite یک موتور دیتابیس رابطه‌ای (Relational) است که بر خلاف بسیاری از سیستم‌های مدیریت دیتابیس (DBMS) مانند MySQL و PostgreSQL، به صورت یک کتابخانه ارائه می‌شود و معمولاً به شکل یک سرویس جداگانه اجرا نمی‌شود. SQLite تمام دیتابیس را در قالب یک فایل (معمولاً با پسوند db. یا sqlite.) ذخیره می‌کند.

ویژگی‌های کلیدی: - فایل‌محور (File-based) - سبک و کم‌هزینه - مناسب برای اپلیکیشن‌های کوچک تا متوسط - سرعت خوب برای خواندن/نوشتن در بسیاری از سناریوها - پشتیبانی از SQL استاندارد و اکثر قابلیت‌های پایه

مثال SQL (یک نمونه ایجاد جدول)

```
) CREATE TABLE users
,id INTEGER PRIMARY KEY
name TEXT NOT NULL
;
```

خروجی:

این کوئری جدول users را ایجاد می‌کند. اگر خطایی رخ ندهد، خروجی خاصی نمایش داده نمی‌شود.

تفاوت SQLite با MySQL و PostgreSQL

SQLite: - بدون نیاز به سرور جداگانه - ذخیره به صورت یک فایل واحد - مناسب برای دستگاه‌های کاربر، موبایل، دسکتاپ و برنامه‌های embedded

MySQL / PostgreSQL: - معمولاً نیازمند سرور هستند - مدیریت همزمانی بهتر در مقیاس‌های بزرگ (به طور کلی) - مناسب برای سیستم‌های تحت وب با ترافیک بالا و معماری چندکاربره

خلاصه:

SQLite برای سبک، محلی و embedded عالی است.
MySQL/PostgreSQL برای مقیاس‌پذیری سرویس‌دهی و معماری کلان‌تر مناسب‌ترند.

مزایا و محدودیت‌ها

مزایا: - راه‌اندازی آسان - عدم نیاز به سرور - مصرف منابع کم - مناسب برای پروژه‌های کوچک و متوسط - قابل حمل (فقط یک فایل دیتابیس)

محدودیت‌ها: - برای بارهای خیلی سنگین و همزمانی‌های بسیار بالا، ممکن است گزینه‌های دیگر بهتر باشند - برخی امکانات پیشرفته (در حد سیستم‌های بزرگ) ممکن است متفاوت یا محدود باشند - مدل همزمانی SQLite (در سطح پایدارسازی داده) متفاوت است و باید در معماری‌های سنگین دقت شود

موارد استفاده رایج

- اپلیکیشن‌های موبایل (اندروید معمولاً SQLite داخلی دارد)
- اپلیکیشن‌های دسکتاپ
- پروژه‌های IoT / embedded
- ابزارهای مدیریتی سبک
- پروتوتایپ سریع یا دیتابیس محلی برای توسعه

نمونه سناریو

یک اپ موبایل برای ثبت سفارش‌ها و وضعیت کاربران، با دیتابیس محلی می‌تواند بسیار مناسب SQLite باشد.

خروجی:

دیتابیس در قالب فایل در دستگاه ذخیره می‌شود.

2. نصب و راه‌اندازی

نصب در ویندوز

1. اگر از پایتون استفاده می‌کنید، احتمالاً SQLite آماده است.
2. یا می‌توانید ابزار SQLite را دانلود کنید:
3. از صفحه رسمی SQLite (بخش دانلود)
4. فایل ZIP را استخراج کنید
5. مسیر پوشه شامل `sqlite3.exe` را به PATH اضافه کنید.

استفاده

در CMD:

```
sqlite3 mydb.sqlite
```

خروجی:

به محیط SQLite می‌روید (علامت `<sqlite` نمایش داده می‌شود).

نصب در لینوکس

در بسیاری از توزیع‌ها SQLite از قبل نصب است یا خیلی ساده نصب می‌شود.

:Debian/Ubuntu

```
sudo apt update  
sudo apt install sqlite3
```

خروجی:

اگر نصب موفق باشد، دستور `sqlite3` قابل اجرا خواهد بود.

نصب در macOS

اگر Xcode Command Line Tools نصب باشد، SQLite معمولاً موجود است. با این دستور چک کنید:

```
sqlite3 --version
```

اگر لازم بود، معمولاً از طریق Homebrew:

```
brew install sqlite
```

خروجی:

نسخه SQLite نمایش داده می‌شود.

استفاده از SQLite در Python

پایتون به طور پیش‌فرض ماژول `sqlite3` دارد.

نمونه کد پایتون: ایجاد دیتابیس و جدول

```
import sqlite3  
  
conn = sqlite3.connect("mydb.sqlite")  
cur = conn.cursor()  
  
cur.execute("""  
    CREATE TABLE IF NOT EXISTS users  
    (id INTEGER PRIMARY KEY  
    )
```

```
name TEXT NOT NULL
(
( " " "

(conn.commit
(conn.close
print("Done")
```

خروجی:

فایل `mydb.sqlite` ساخته می‌شود و جدول `users` ایجاد می‌گردد (اگر وجود نداشته باشد).

3. مفاهیم پایه دیتابیس

دیتابیس چیست؟

دیتابیس مکانی برای ذخیره و مدیریت داده‌ها به شکل ساختاریافته است. در دیتابیس رابطه‌ای (Relational)، داده‌ها معمولاً در جدول‌ها ذخیره می‌شوند.

جدول (Table)

جدول شبیه یک صفحه اکسل است: - ستون‌ها ویژگی‌های داده را نگه می‌دارند - سطرها داده‌های واقعی (رکوردها) را تشکیل می‌دهند

مثال

```
) CREATE TABLE users
,id INTEGER PRIMARY KEY
,name TEXT
age INTEGER
; (
```

خروجی:

جدول `users` ایجاد می‌شود.

سطر و ستون

- ستون (Column): نوع و اطلاعات هر فیلد
- سطر (Row): یک رکورد کامل

مثلاً در جدول `users` - ستون‌ها: `id` , `name` , `age` - هر کاربر یک سطر است.

کلید اصلی (Primary Key)

کلید اصلی یعنی هر رکورد یک شناسه یکتا دارد.

در SQLite:

```
id INTEGER PRIMARY KEY
```

نکته: در SQLite معمولاً اگر نوع `INTEGER` باشد، می‌تواند رفتار مخصوصی برای مدیریت مقاداردهی خودکار (`ROWID`) داشته باشد.

خروجی:

SQLite تضمین می‌کند مقدار `id` برای هر سطر یکتا باشد.

انواع داده در SQLite

SQLite سیستم تایپ سخت‌گیرانه مثل بعضی DBMS ها ندارد. اما انواع رایج: `INTEGER` - `REAL` (اعشاری) - `TEXT` (متنی) - `BLOB` (داده باینری) - `NULL` (مقدار تهی)

مثال

```
) CREATE TABLE products
,id INTEGER PRIMARY KEY
,title TEXT NOT NULL
,price REAL
,image BLOB
; (
```

خروجی:
جدول ساخته می‌شود.

4. دستورات پایه SQL در SQLite

در این بخش دستورات اصلی SQL را با مثال می‌بینیم.

CREATE TABLE

برای ساخت جدول.

مثال

```
CREATE TABLE users
(id INTEGER PRIMARY KEY
,username TEXT NOT NULL
,email TEXT UNIQUE
,created_at TEXT
);
```

توضیح خروجی:
اگر موفق باشید، جدول ایجاد می‌شود. اگر جدول وجود داشته باشد ممکن است خطا بگیرند مگر اینکه از `IF NOT EXISTS` استفاده کنید.

INSERT

برای افزودن داده به جدول.

مثال

```
INSERT INTO users (username, email, created_at)
VALUES ('amir', 'amir@example.com', '2026-01-10')
```

توضیح خروجی:

رکورد جدید وارد جدول می‌شود. در محیط `sqlite3` معمولاً یک پیام مثل `Run` یا `changes` ممکن است دیده شود.

روش بهتر (چند رکورد):

```
INSERT INTO users (username, email, created_at)
VALUES
    ('sara', 'sara@example.com', '2026-01-11')
;('reza', 'reza@example.com', '2026-01-12')
```

SELECT

برای خواندن داده‌ها.

مثال

```
SELECT id, username, email
;FROM users
```

توضیح خروجی:

لیستی از سطرهای جدول `users` نمایش داده می‌شود.

UPDATE

برای تغییر داده‌ها.

مثال

```
UPDATE users
SET email = 'amir.new@example.com'
;WHERE username = 'amir'
```

توضیح خروجی:

فقط سطرهایی که شرط `WHERE` را برآورده کنند آپدیت می‌شوند.

DELETE

برای حذف داده‌ها.

مثال

```
DELETE FROM users  
; 'WHERE username = 'reza
```

توضیح خروجی:

رکوردهای مطابق شرط حذف می‌شوند.

احتیاط: اگر `WHERE` نگذارید، ممکن است کل جدول خالی شود.

5. کار با شرط‌ها

WHERE

برای فیلتر کردن نتایج.

مثال

```
* SELECT  
FROM users  
; WHERE age > 20
```

توضیح خروجی:

فقط کاربرانی که `age` بزرگ‌تر از ۲۰ است نمایش داده می‌شوند.

AND / OR

برای ترکیب شرطها.

مثال با AND

```
* SELECT  
FROM users  
;WHERE age >= 18 AND age <= 30
```

خروجی:

کاربرانی با سن بین ۱۸ تا ۳۰.

مثال با OR

```
* SELECT  
FROM users  
;'WHERE username = 'amir' OR username = 'sara
```

خروجی:

کاربرانی که نام کاربری‌شان یکی از این دو مقدار باشد.

LIKE

برای جستجوی الگوهای متنی. - % یعنی هر تعداد کاراکتر - _ یعنی یک کاراکتر

مثال

```
* SELECT  
FROM users  
;'WHERE email LIKE '%@example.com
```

توضیح خروجی:
همه ایمیل‌هایی که دامنه‌شان example.com است.

BETWEEN

برای بازه (اعدادی یا قابل مقایسه).

مثال

```
* SELECT  
FROM orders  
;WHERE total BETWEEN 100 AND 500
```

خروجی:
سفارش‌هایی با مبلغ بین ۱۰۰ تا ۵۰۰.

IN

برای بررسی عضویت در مجموعه‌ای از مقادیر.

مثال

```
* SELECT  
FROM users  
;WHERE id IN (1, 3, 5)
```

خروجی:
سطرهایی که id برابر یکی از این اعداد باشد.

6. مرتب‌سازی و محدود کردن نتایج

ORDER BY

برای مرتب‌سازی نتایج بر اساس یک یا چند ستون.

مثال

```
* SELECT  
FROM users  
;ORDER BY created_at DESC
```

توضیح خروجی:
جدول بر اساس `created_at` از جدید به قدیم مرتب می‌شود.

افزودن ترتیب صعودی:

```
;ORDER BY age ASC
```

LIMIT

برای محدود کردن تعداد رکوردها.

مثال

```
* SELECT  
FROM users  
ORDER BY id DESC  
;LIMIT 10
```

توضیح خروجی:
۱۰ رکورد آخر بر اساس `id` نمایش داده می‌شوند.

ترکیب `ORDER BY + LIMIT` در اکثر پروژه‌ها بسیار رایج است.

7. توابع مهم SQLite

COUNT

تعداد رکوردها.

```
SELECT COUNT(*) AS total_users  
;FROM users
```

توضیح خروجی:
تعداد کل کاربران.

SUM

جمع مقادیر عددی.

مثال

```
SELECT SUM(total) AS sum_orders  
;FROM orders
```

توضیح خروجی:
جمع مبلغ کل سفارش‌ها.

AVG

میانگین.

```
SELECT AVG(total) AS avg_order  
;FROM orders
```

خروجی:
میانگین مبلغ سفارش‌ها.

MIN / MAX

کمینه / بیشینه.

```
SELECT
    ,MIN(total) AS min_order
    ,MAX(total) AS max_order
;FROM orders
```

خروجی:

کمترین و بیشترین مبلغ سفارش‌ها.

8. ارتباط بین جدول‌ها

Foreign Key

کلید خارجی رابطه بین دو جدول را برقرار می‌کند. یعنی یک ستون در جدول A به کلید اصلی جدول B اشاره می‌کند.

مثال: `orders.user_id` به `users.id` اشاره می‌کند.

ایجاد جدول با Foreign Key

```
) CREATE TABLE orders
    ,id INTEGER PRIMARY KEY
    ,user_id INTEGER NOT NULL
    ,total REAL NOT NULL
    ,created_at TEXT
FOREIGN KEY (user_id) REFERENCES users(id)
; (
```

توضیح خروجی:

SQLite اجازه می‌دهد اما برای فعال بودن enforcement در SQLite باید مطمئن شوید که `foreign_keys` روشن است (در SQL می‌توانید اجرا کنید).

فعال سازی:

```
;PRAGMA foreign_keys = ON
```

JOIN

برای ترکیب داده‌ها از چند جدول بر اساس شرط رابطه.

INNER JOIN

فقط رکوردهای دارای تطابق را نشان می‌دهد.

مثال

```
SELECT
    ,u.username
    ,o.total
    o.created_at
FROM users u
;INNER JOIN orders o ON o.user_id = u.id
```

توضیح خروجی:

کاربرانی که حداقل یک سفارش دارند (و تطابق دارند).

LEFT JOIN

همه رکوردهای سمت چپ را نشان می‌دهد، حتی اگر تطابق وجود نداشته باشد.

```
SELECT
    ,u.username
    ,o.total
    o.created_at
```

```
FROM users u  
;LEFT JOIN orders o ON o.user_id = u.id
```

توضیح خروجی:

اگر کاربری سفارش نداشته باشد، ستون‌های مربوط به سفارش NULL می‌شوند.

9. ایندکس‌ها (Indexes)

چرا ایندکس مهم است؟

ایندکس مثل یک فهرست برای دیتابیس عمل می‌کند. بدون ایندکس، دیتابیس باید داده‌ها را برای یافتن رکوردها تماماً اسکن کند. با ایندکس، معمولاً سریع‌تر می‌رسد.

ایجاد ایندکس

مثال

اگر در کوئری زیاد از `WHERE username = ...` استفاده می‌کنید:

```
CREATE INDEX idx_users_username  
ON users(username)
```

خروجی:

ایندکس ایجاد می‌شود.

تاثیر در سرعت کوئری

- برای کوئری‌های فیلتر/جستجو (`WHERE` , `JOIN`) معمولاً سرعت بهتر می‌شود.
- اما برای عملیات نوشتن (`INSERT` , `UPDATE` , `DELETE`) چون ایندکس باید به‌روزرسانی شود ممکن است کمی کندتر شود.

نکته عملی:

ایندکس را فقط برای ستون‌هایی بسازید که واقعاً زیاد در شرط‌ها یا `join`ها استفاده می‌شوند.

10. تراکنش‌ها (Transactions)

SQLite از ACID پشتیبانی می‌کند و تراکنش‌ها باعث می‌شوند چند عملیات به صورت یک واحد انجام شوند.

BEGIN

شروع تراکنش.

```
;BEGIN TRANSACTION
```

COMMIT

ثبت نهایی تغییرات.

```
;COMMIT
```

ROLLBACK

برگرداندن تغییرات در صورت خطا یا تصمیم به لغو.

```
;ROLLBACK
```

مثال عملی

فرض کنید می‌خواهیم همزمان یک کاربر اضافه کنیم و چند سفارش مرتبط با آن.

```
;PRAGMA foreign_keys = ON
```

```
;BEGIN TRANSACTION
```

```
INSERT INTO users (username, email, created_at)  
VALUES ('mahsa', 'mahsa@example.com', '2026-01-15')
```

```
-- فرض: user_id همان id جدید است (در برنامه معمولاً از last_insert_rowid
INSERT INTO orders (user_id, total, created_at)
(SELECT id FROM users WHERE username = 'mahsa'), 250.50, '2026-01-16')

;COMMIT
```

توضیح خروجی:

اگر همه چیز درست باشد، هر دو جدول به روزرسانی می شوند.
اگر خطایی رخ دهد، با ROLLBACK می توانید همه تغییرات را برگردانید.

11. مثال پروژه کوچک

در این بخش یک پروژه کوچک می سازیم: مدیریت کاربران با: - جدول users - جدول orders - چند کوئری کاربردی

ساخت دیتابیس مدیریت کاربران

1) ایجاد جدول users

```
) CREATE TABLE users
,id INTEGER PRIMARY KEY
,username TEXT NOT NULL UNIQUE
,email TEXT NOT NULL UNIQUE
created_at TEXT NOT NULL
;(
```

خروجی:

جدول users ایجاد می شود.

2) ایجاد جدول orders

```
) CREATE TABLE orders
,id INTEGER PRIMARY KEY
```

```
,user_id INTEGER NOT NULL
    ,total REAL NOT NULL
    ,'status TEXT NOT NULL DEFAULT 'pending'
    ,created_at TEXT NOT NULL
FOREIGN KEY (user_id) REFERENCES users(id)
; (
```

خروجی:

جدول `orders` ایجاد می‌شود.

وارد کردن داده نمونه

```
INSERT INTO users (username, email, created_at)
VALUES
    ,('ali', 'ali@example.com', '2026-02-01')
    ,('sara', 'sara@example.com', '2026-02-03')
    ;('reza', 'reza@example.com', '2026-02-05')
```

```
INSERT INTO orders (user_id, total, status, created_at)
VALUES
    , 'paid', '2026-02-10' ,('SELECT id FROM users WHERE username = 'ali'))
    , 'pending', '2026-02-11' ,('SELECT id FROM users WHERE username = 'ali'))
    , 'paid', '2026-02-12' ,('SELECT id FROM users WHERE username = 'sara'))
```

توضیح خروجی:

سه کاربر و سه سفارش نمونه ذخیره می‌شوند.

چند کوئری کاربردی

کوئری 1: لیست کاربران

```
SELECT id, username, email, created_at
FROM users
;ORDER BY created_at DESC
```

خروجی:

جدول کاربران مرتب شده نمایش داده می شود.

کوئری 2: دریافت سفارش های هر کاربر (JOIN)

```
SELECT
    ,u.username
    ,o.total
    ,o.status
    o.created_at
FROM users u
INNER JOIN orders o ON o.user_id = u.id
;ORDER BY o.created_at DESC
```

خروجی:

هر سفارش به همراه نام کاربر نمایش داده می شود.

کوئری 3: کاربران بدون سفارش (LEFT JOIN + شرط)

کاربرانی که سفارش ندارند.

```
SELECT
    ,u.username
    ,u.email
    u.created_at
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
;WHERE o.id IS NULL
```

خروجی:

لیست کاربرانی که هیچ سفارشی ثبت نکرده اند.

کوئری 4: مجموع سفارش‌های هر کاربر (GROUP BY)

```
SELECT
    u.username
    ,COUNT(o.id) AS orders_count
    ,SUM(o.total) AS orders_sum
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id
;ORDER BY orders_sum DESC
```

خروجی:

به ازای هر کاربر: - تعداد سفارش‌ها - مجموع مبلغ سفارش‌ها
نمایش داده می‌شود.

12. بهترین روش‌ها (Best Practices)

طراحی مناسب جدول

- از Primary Key برای هر جدول استفاده کنید.
- ستون‌هایی که لازم دارند NOT NULL شوند.
- از UNIQUE برای فیلدهایی که باید یکتا باشند استفاده کنید (مثل email).
- نوع داده‌ها را تا حد امکان درست انتخاب کنید (INTEGER برای عدد صحیح، REAL برای اعشاری و ...).

مثال

```
CREATE TABLE users
(
    id INTEGER PRIMARY KEY
    ,username TEXT NOT NULL UNIQUE
    ,email TEXT NOT NULL UNIQUE
    ,created_at TEXT NOT NULL
);
```

استفاده از ایندکس

- ایندکس را برای ستون‌هایی بسازید که زیاد در `WHERE` ، `JOIN` و `ORDER BY` استفاده می‌شوند.
- اما بی‌دلیل ایندکس زیاد نسازید.

مثال

```
CREATE INDEX idx_orders_user_id  
ON orders(user_id)
```

توضیح:

اگر زیاد `join` می‌زنید روی `orders.user_id` این ایندکس کمک می‌کند.

جلوگیری از کوئری‌های سنگین

- در کوئری‌های بزرگ از `SELECT *` تا حد امکان استفاده نکنید؛ فقط ستون‌های لازم را انتخاب کنید.
- نتایج را با `LIMIT` مدیریت کنید.
- برای فیلترها حتماً `WHERE` مناسب بگذارید.
- از خواندن غیرضروری داده‌ها (`scan` بزرگ بدون شرط) جلوگیری کنید.

مثال بهتر نسبت به `SELECT *`

```
SELECT id, username  
FROM users  
WHERE username LIKE 'a'
```

خروجی:

فقط ستون‌های لازم و رکوردهای مرتبط برمی‌گردد.

13. منابع برای یادگیری بیشتر

منابع رسمی

- وبسایت رسمی SQLite:
<https://www.sqlite.org/>
- راهنمای SQL و مستندات دستورات و قابلیت‌ها در وبسایت SQLite

منابع آموزشی

- دوره‌ها و آموزش‌های SQLite3 در پلتفرم‌های آموزشی (Coursera/Udemy/...)
- مستندات و مثال‌های پایتون برای `sqlite3` در سایت‌های برنامه‌نویسی معتبر

پیشنهاد مسیر یادگیری

1. مفاهیم پایه: جدول، سطر، کلیدها
2. SQL پایه: `DELETE` , `UPDATE` , `SELECT` , `INSERT` , `CREATE`
3. شرط‌ها: `BETWEEN` , `IN` , `LIKE` , `WHERE`
4. ترکیب جدول‌ها: `JOIN`
5. بهینه‌سازی: ایندکس و محدود کردن نتایج
6. اصول دیتابیس: تراکنش‌ها و مدیریت خطا

پیوست: نمونه SQL تکمیلی (جمع‌بندی سریع)

1) ایجاد جدول

```
) CREATE TABLE sample
,id INTEGER PRIMARY KEY
name TEXT NOT NULL
; (
```

(2) افزودن داده

```
;INSERT INTO sample (name) VALUES ('test')
```

(3) خواندن داده

```
;SELECT * FROM sample
```

(4) تغییر داده

```
;UPDATE sample SET name = 'updated' WHERE id = 1
```

(5) حذف داده

```
;DELETE FROM sample WHERE id = 1
```

پایان سند